

Technical Design — Ingestion Module

Theme & Epic Generation · Ingestion

1. Purpose & scope

The ingestion module converts **historical IDMT Engagement Request tickets** into a trusted corpus that the data-science (generation) module retrieves from. For every eligible ticket it produces:

1. a **condensed, durable record** of the ticket's content (system of record), and
2. each linked **Theme's** title, description, and **Value Stream** — the Business Architect's recorded Value Stream for that ticket, used as retrieval precedent.

Ingestion is **offline/batch** and contains no runtime generation logic. It writes to **Cosmos** (the system of record) and **idp_teg_data** (the retrieval index). The governed Value Stream / Stage / L2 / L3 catalogue is **not** produced here — it is the org's gold data in the **Azure SQL DB**, consumed as-is.

2. Flow

```
STAGE 0 – Ticket identification (Neo4j 5-filter funnel)
  Neo4j JIRA graph → one Cypher query (L2→L6 funnel) → list of usable IDMT ticket keys

STAGE 1 – Per-ticket ingestion (run once per identified key)
  IDMT Engagement Request (Jira)
    → fetch ER + linked Themes
    → attachment extraction (no idea-card detection)
    → raw text assembly (description + attachments → ~24k-token budget)
    → Condense (LLM) → business-context fields + rawText
    → Theme extraction → title, description, Value Stream (from Jira)
    → write Cosmos (ER doc + Theme docs) + upsert idp_teg_data (searchText + content_vector)
```

3. Ticket identification (which tickets we ingest)

A **distinct first stage, upstream of the per-ticket pipeline**. We do **not** ingest every IDMT ticket — we first identify the **usable Value-Stream cohort** and produce a list of their keys. The per-ticket pipeline (Stage 1) then runs once per identified key.

Identification runs against the **Neo4j JIRA graph** (env: `NEO4J_URI / USER / PASSWORD / DATABASE`) as a **single Cypher query** implementing a 5-filter funnel (L2→L6). A ticket survives to the cohort only if **all** hold:

filter	rule
L2 — is an IDMT Engagement Request, recent	key starts with IDMT-, issueType = "Engagement Request", creationDateEpoch ≥ since (default 2023-01-01)
L3 — not in a dead status	status NOT IN {Cancelled, Blocked, New Request}
L4 — is implemented by a linked issue	the IDMT has ≥1 inbound "implemented by" link — i.e. a Theme <i>implements</i> it. (From the IDMT's side this relationship is inward; the same link is outward "implements" on the Theme. We read it from the IDMT, so it appears inward.)
L5 — the linked issue is a Theme	the linked key resolves to a JIRA node with issueType = "Theme"
L6 — the Theme carries a Value Stream	the Theme's businessValueStreams matches ...{VSR\d+} (a valid Value Stream id is present)

The query returns the **distinct ER keys** that pass all five; that key set is the cohort the batch ingestion run consumes.

Status note. The identification status exclusion is {Cancelled, Blocked, New Request} — the funnel rejects whole tickets that were dropped, on hold, or never started.

4. Attachment extraction

The business content comes from the ticket's **attachments**, fetched directly — there is **no idea-card detection**; every supported attachment is extracted.

- **Supported formats:** .pdf , .pptx , .docx .
- Legacy binary .ppt / .doc and image-only files would yield no text — but the EDA found **none** of these in the corpus, so all attachments in scope are text-extractable. No OCR.

5. Raw text assembly

The Jira **description** and the extracted attachment text are concatenated into a single **raw text** blob, greedily packed into a **~24k-token budget**. The description is part of that budget — there is no separate budget for it. Highest-priority content is never displaced or truncated; the token budget is the only cap.

6. Condense (LLM)

A single LLM pass extracts structured business context from the raw text, so downstream steps don't re-process it. It produces the LLM-derived fields stored on the IDMT document (§7), and the raw text is carried through as rawText .

7. Fields extracted

We extract content directly from Jira (and the LLM condense pass). We do **not** extract or store Epics, Stages, L2/L3 capabilities, or Business Needs — only the Theme's title, description, and Value Stream.

7.1 IDMT Engagement Request

field	source
key , <code>sourceId</code>	Jira issue key + internal id
description	Jira description
summary	ticket title
creationDate , <code>insightsTime</code>	source created / last-updated dates
status	Jira status (stored on the index doc)
rawText	description + attachment text, packed to ~24k tokens (\$5)
businessSummary	LLM-generated business summary (condense)
keyTerms	domain terms & acronyms (condense)
businessProblem	business problem / pain point (condense)
businessCapability	desired capability / outcome (condense)
stakeholders	stakeholder groups (condense)
systemsAndProducts	referenced systems, platforms, products (condense)

7.2 Theme

field	source
key , <code>sourceId</code>	Jira issue key (GROUP-####) + internal id
summary	Theme title
description	Theme description
valueStream	the Theme's Business Value Stream field, formatted " <code><name> {id}</code>", taken as-is — no fuzzy matching, no LLM, no catalogue re-resolution</code>
creationDate , <code>insightsTime</code>	Theme created / last-updated dates

8. Storage schema

8.1 Cosmos — Engagement Request document

field	description
id	document uuid
key	Jira issue key, e.g. IDMT-#### (mutable business key)
sourceId	stable Jira internal id (e.g. 3364549); stable across IDMT-key changes
source	origin system, e.g. Jira
entityType	ENGAGEMENTREQUEST
createdAt / createdBy	Cosmos creation date / actor
lastModifiedAt / lastModifiedBy	Cosmos modification date / actor
parentRef	sourceId (an ER has no parent)
properties	nested object — extracted business context (below)

properties: description, summary, creationDate, insightsTime, businessSummary, keyTerms, businessProblem, businessCapability, stakeholders, systemsAndProducts, rawText.

8.2 Cosmos — Theme document

field	description
id	document uuid
key	Jira issue key (GROUP-####)
sourceId	stable Jira internal id
source	origin system, e.g. Jira
entityType	THEME
createdAt / createdBy	Cosmos creation date / actor
lastModifiedAt / lastModifiedBy	Cosmos modification date / actor
parentRef	the parent IDMT ticket's sourceId
properties	nested object (below)

properties: summary (Theme title), description, valueStream (Value Stream linked to the Theme), creationDate, insightsTime.

Themes are **separate documents**, found via parentRef — not embedded as a themes[] array on the IDMT document.

8.3 AI Search index (idp_teg_data) — retrieval-only

Holds the historical Engagement-Request documents for retrieval.

field	description
key	Jira issue key, e.g. IDMT-#### (mutable business key)
sourceId	stable Jira internal id (stable across IDMT-key changes)
entityType	ENGAGEMENTREQUEST
status	Jira status (Cancelled / To Do / In Progress ...)
searchText	businessSummary + businessProblem + businessCapability + keyTerms + stakeholders + systemsAndProducts
content_vector or	vectorized searchText

The index returns ranked ids; the full ticket details are read from Cosmos at query time.

8.4 Azure SQL DB — governed catalogue (consumed, not produced)

The Value Stream / Stage / L2 / L3 catalogue is the org's **gold data in Azure SQL**, read as-is at runtime. Ingestion does **not** define or store this catalogue.

9. Rules & conventions

- **Content is read directly from Jira** — the Theme's Value Stream is taken as-is from its Business Value Stream field; no fuzzy matching, no LLM, no catalogue re-resolution.
- **We store only the Theme's title, description, and Value Stream** — no Epics, Stages, L2/L3, or Business Needs are extracted or stored.
- **The index is retrieval-only** — searchText + content_vector ; no labels or signals stored in it.
- **The Value Stream catalogue is Azure SQL gold data**, consumed as-is — not redefined or stored here.
- **Unit tests make no live Jira / Azure / LLM calls** — clients are injected (fakes).